

PROJETO I

1. INTRODUÇÃO

O primeiro projeto consistia em aplicar algumas das estruturas lineares abordadas ao longo dos primeiros meses na disciplina Estrutura de Dados. Este relatório detalha a implementação de um programa projetado para processar arquivos disponibilizados pelo professor no formato XML. O problema central é formado por duas tarefas principais: primeiro, validar a estrutura de aninhamento das tags do arquivo XML e, então, interpretar os dados contidos para calcular a área de um "componente conexo" dentro de uma matriz binária.

O contexto da aplicação simula um robô aspirador que opera em um cenário definido por uma matriz, onde o valor '1' representa uma área a ser limpa e '0' uma área que deve ser ignorada. A partir de uma posição inicial (x,y), o programa precisa calcular o tamanho total da área contínua (conexa) que o robô pode alcançar. Para solucionar os problemas, foram utilizadas estruturas de dados lineares: uma pilha para a validação do XML e uma fila para o cálculo da área, implementando o algoritmo de Busca em Largura (BFS).

2. DESCRIÇÃO DOS ALGORITMOS PRINCIPAIS

A solução foi dividida em duas partes lógicas, cada uma abordando um dos problemas propostos.

2.1. VALIDAÇÃO DA ESTRUTURA DO ARQUIVO XML

Para verificar se as tags em um arquivo XML estão corretamente aninhadas e fechadas, foi implementado um algoritmo baseado na estrutura de dados de pilha, que segue o princípio LIFO. Partimos da ideia de que a última tag aberta deve ser a primeira a ser fechada.

A lógica implementada na função `validar_xml` segue os seguintes passos:

1. O programa percorre o conteúdo do arquivo XML caractere por caractere.
2. Ao encontrar um caractere de abertura de tag (<), o algoritmo verifica o próximo caractere.

3. Tag de Abertura: Se não for uma barra (/), o nome da tag (ex: "cenario", "nome") é extraído e realizado um *push*, deixando registrado que essa tag foi aberta e está aguardando seu fechamento.
4. Tag de Fechamento: Se o caractere for uma barra (/), o nome da tag de fechamento é extraído. O algoritmo então verifica o topo da pilha:
 - a. Erro de Aninhamento: Se o nome da tag de fechamento for diferente do nome no topo da pilha, significa que uma tag interna não foi fechada antes de uma externa. A função retorna *false*.
 - b. Erro de Tag Extra: Se a pilha estiver vazia, significa que uma tag está sendo fechada sem nunca ter sido aberta. A função retorna *false*.
 - c. Fechamento Correto: Se os nomes coincidirem, a tag no topo da pilha é desempilhada (pop), indicando que o par de tags foi corretamente fechado.
5. Verificação Final: Após percorrer todo o arquivo, a pilha deve estar vazia. Se ainda houver elementos, significa que uma ou mais tags foram abertas, mas nunca fechadas. Nesse caso, a função retorna *false*.

Se nenhuma dessas condições de erro for encontrada ao longo de toda a análise, a função retorna *true*, indicando que o XML é válido estruturalmente.

A figura abaixo demonstra o estado da pilha durante o processamento de um trecho XML válido: <cenario><nome></nome></cenario>. Ao final, a pilha está vazia, indicando sucesso.

Figura 1: Ilustração do Algoritmo de Validação com Pilha

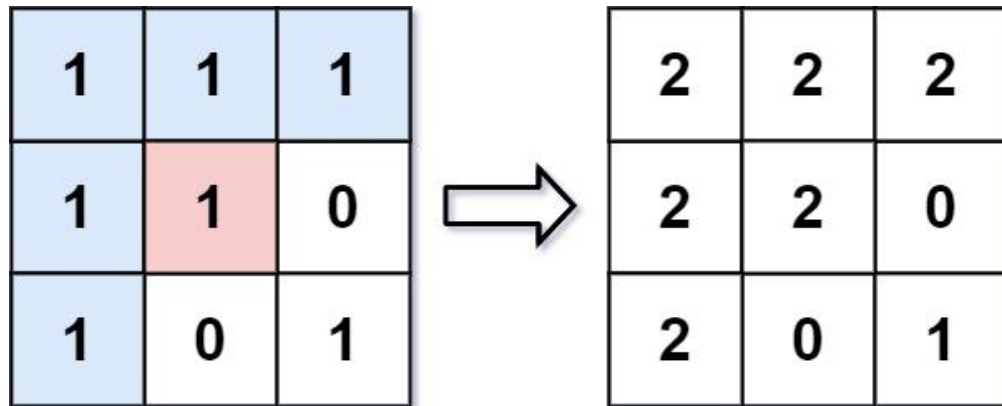
Texto Processado	Ação	Estado da Pilha
<cenario>	Push "cenario"	["cenario"]
<nome>	Push "nome"	["cenario", "nome"]
</nome>	Pop "nome"	["cenario"]
</cenario>	Pop "cenario"	[] (Vazia)

2.2. CÁLCULO DE ÁREA DO COMPONENTE CONEXO

Para a determinação da área a ser limpa, é necessário identificar o componente conexo ao qual o robô pertence. Para isso, foi implementado o clássico

algoritmo Flood Fill (também conhecido como preenchimento por inundação), que utiliza uma abordagem de Busca em Largura (BFS) para explorar a região.

Figura 2: Ilustração do Algoritmo Flood Fill



Uma vez que o XML é validado, o programa extrai os dados de cada cenário (dimensões da matriz, ponto inicial e a própria matriz) para a classe Cenário. O desafio seguinte é calcular a área do componente conexo a partir de um ponto inicial (x, y). Um componente conexo é o conjunto de todos os pontos com valor '1' que podem ser alcançados a partir do ponto inicial, movendo-se apenas para cima, para baixo, para a esquerda ou para a direita (vizinhança-4).

Para essa tarefa, foi utilizado o algoritmo de Busca em Largura, que emprega uma fila, estrutura do tipo FIFO. O BFS é perfeito para explorar a matriz em "camadas", garantindo que todos os vizinhos de um ponto sejam visitados antes de se aprofundar na busca.

A lógica da função *calcular_area_componente_conexo* é a seguinte:

1. Inicialização:
 - a. Uma matriz visitado (do tipo `vector<vector<bool>>`), com as mesmas dimensões do cenário, é criada e preenchida com false. Ela serve para marcar os pontos já visitados, evitando loops e recontagens.
 - b. Uma fila é inicializada para armazenar as coordenadas (x, y) dos pontos a serem explorados.
 - c. A área é inicializada com 0.
2. Ponto de Partida: O ponto inicial (cenario.x, cenario.y) é verificado. Se ele estiver fora dos limites da matriz ou se o seu valor for '0', a área é 0 e o algoritmo termina. Caso contrário, o ponto é adicionado à fila e marcado como visitado.

3. Loop de Exploração (BFS): O algoritmo entra em um laço que continua enquanto a fila não estiver vazia.
 - a. A coordenada (x, y) do primeiro elemento da fila é removida (pop).
 - b. A área é incrementada em 1, pois um ponto válido do componente foi processado.
 - c. Os quatro vizinhos do ponto (x, y) são examinados: (x-1, y), (x+1, y), (x, y-1) e (x, y+1).
 - d. Para cada vizinho, o algoritmo verifica três condições:
 - i. Está dentro dos limites da matriz.
 - ii. Seu valor na matriz do cenário é '1'.
 - iii. Ainda não foi visitado (seu valor em visitado é *false*).
 - e. Se todas as condições forem atendidas, o vizinho é marcado como visitado e adicionado (push) ao final da fila para ser processado posteriormente.
4. Resultado: Quando a fila fica vazia, significa que todos os pontos alcançáveis a partir do ponto inicial foram visitados. O valor final da variável area é o tamanho do componente conexo, que é então retornado.

Exemplo prático: Considere a matriz 4x4 abaixo e o ponto inicial (1,1).

```

0 0 1 0
0 1 1 0
0 1 0 1
1 1 0 0

```

Figura 3: Ilustração do Algoritmo de Cálculo de Área com Fila (BFS)

Ação	Fila	Área	Células Visitadas [(x,y)]
Inserir (1,1)	[(1,1)]	0	[(1,1)]
Remover (1,1), Adicionar vizinhos (1,2) e (2,1)	[(1,2), (2,1)]	1	[(1,1), (1,2), (2,1)]
Remover (1,2), Nenhum vizinho novo	[(2,1)]	2	[(1,1), (1,2), (2,1)]
Remover (2,1), Adicionar vizinho (3,1)	[(3,1)]	3	[(1,1), (1,2), (2,1), (3,1)]
Remover (3,1), Adicionar vizinho (3,0)	[(3,0)]	4	[(1,1), (1,2), (2,1), (3,1), (3,0)]
Remover (3,0), Nenhum vizinho novo	[] (Vazia)	5	[(1,1), (1,2), (2,1), (3,1), (3,0)]

3. CONCLUSÃO E DIFICULDADES ENCONTRADAS

O desenvolvimento do projeto permitiu a aplicação prática de estruturas de dados fundamentais na resolução de problemas concretos de validação de dados e análise de matrizes. Alguns desafios foram encontrados. A principal dificuldade foi no desenvolvimento da função *calcular_area_componente_conexo*. A criação desta função, exigiu atenção a vários detalhes críticos, tornando-a a parte mais complexa da nossa implementação.

Os principais desafios específicos dentro desta função foram:

- **Lógica do Algoritmo (Flood Fill/BFS):** Garantir que a exploração da matriz seguisse corretamente a lógica da Busca em Largura, visitando todos os pontos conexos sem repetições ou omissões.
- **Gerenciamento de Estado:** A utilização correta da matriz visitado foi fundamental para evitar loops infinitos, que ocorreriam se o algoritmo tentasse re-visitar pontos já processados.
- **Tratamento de Condições de Contorno:** A verificação dos quatro vizinhos de cada célula exigiu um cuidado especial para não acessar posições fora dos limites da matriz (por exemplo, uma coordenada negativa ou maior que a dimensão da matriz).
- **Nuances Técnicas de Tipos de Dados:** Foi preciso prever e tratar o risco de underflow com o tipo de dado *size_t*. Subtrair 1 de uma coordenada de valor 0 causaria um erro. A solução foi converter temporariamente as coordenadas para int antes de realizar as operações de vizinhança.

Adicionalmente, a análise manual do XML, apresentou-se como um processo trabalhoso e levemente suscetível a erros. Apesar dos desafios, o projeto foi concluído com sucesso, atingindo os objetivos propostos de validação e processamento dos cenários.

4. REFERÊNCIAS

Algoritmo de preenchimento por inundação explicado. Disponível em: <<https://www.freecodecamp.org/portuguese/news/algoritmo-de-preenchimento-por-inundacao-explicado/>>. Acesso em: 14 out. 2025.

Busca em largura (BFS) num grafo. Disponível em:
<https://www.ime.usp.br/~pf/algoritmos_para_grafos/aulas/bfs.html>. Acesso em: 14 out. 2025.

Flood Fill Algorithm. Disponível em:
<<https://www.geeksforgeeks.org/dsa/flood-fill-algorithm/>>. Acesso em: 14 out. 2025.